SAT radar: browser-native text-to-speech (SpeechSynthesis)

Ring: Adopt (for direct voice-output use cases; see scope note)
Category: Browser platform capability, not an installable tool
Standardization status: W3C Community Group Draft Report (not yet a full Recommendation, as of July 2026)
Zero-configuration availability: Built into the browser; no install, no account, no API key

---

Scope note: two halves, one name, very different stories

The Web Speech API bundles two interfaces that behave nothing alike:

- `SpeechSynthesis` (text → spoken audio) is what this entry evaluates. It runs through the browser's local, OS-provided voice engines and works fully offline.
- `SpeechRecognition` (microphone → text) is a different beast entirely: in Chrome, audio is sent to Google's servers for processing. Cloud-dependent, no offline path, a completely different sovereignty posture.

Treating these as one capability with one rating would be a UC-lens error. This entry covers `SpeechSynthesis` only. `SpeechRecognition` deserves its own separate radar entry if a UC deployment ever needs speech input, not output.

What it is

`SpeechSynthesis`, exposed as `window.speechSynthesis`, converts text into spoken audio using whatever voices the operating system already has installed. A developer creates a `SpeechSynthesisUtterance` (the text plus pitch, rate, and volume settings) and calls `speechSynthesis.speak()`. No server round-trip, no third-party API, no signup.

This is the exact mechanism Cboard uses for its core spoken output, confirmed directly from its own documentation: *"The app uses the browser's Speech Synthesis API to generate speech when a symbol is clicked."* Evaluating this API is evaluating the thing that makes Cboard's entire premise work.

Universal Cake criteria

Criterion	Rating	Notes
Zero cost	Strong	Built into every modern browser; no account, key, or subscription
Accessibility / disability	Strong for AAC, Weak if conflated with screen readers	See dedicated section below; this distinction matters enough to break out separately

Criterion	Rating	Notes
Device reach	Partial	Near-universal on modern desktop browsers; Firefox for Android has no synthesis support at all; iOS requires a user gesture to trigger playback
Multilingual	Partial	Entirely dependent on which voices the operating system ships; no guarantee any given language has a local voice on a given device
Data sovereignty	Strong (for this half only)	Synthesis runs locally through OS voice engines; works fully offline; do not extend this rating to SpeechRecognition
Standardization maturity	Weak	Still a W3C Community Group Draft Report, not a finished Recommendation; cross-engine behaviour can still drift

Strengths

Zero-dependency voice output. No cloud TTS service, no API key, no per-character billing, no network requirement once the page has loaded. For a project like UC that weighs sovereignty heavily, this is about as clean as voice output gets: the browser already has the capability, for free, running locally.

Genuinely offline-capable. Confirmed directly: synthesis "runs locally without needing an internet connection," in contrast to the API's SpeechRecognition sibling, which requires connectivity to reach a cloud recognition service. This asymmetry is the single most important fact in this entry.

Broad modern-browser coverage. Supported in Chrome (from version 33), Safari (from version 7 on macOS, iOS 7), Firefox desktop (from version 49), Opera (Chromium-based, tracks Chrome), and Samsung Internet (from 5.0, via the Android TTS engine). This covers the large majority of desktop and mobile browsing.

No specialized infrastructure required. A developer needs nothing beyond `if ('speechSynthesis' in window)` as a feature check. No SDK, no build step, no vendor account.

The right fit for AAC specifically. Where the entire point of a tool is direct voice output triggered by user action, as with Cboard, this API is precisely the correct mechanism: local, free, no dependency on a third party staying in business or a subscription staying paid.

Areas needing work

Firefox for Android has no synthesis support. Not a bug, a gap: Firefox for Android does not implement the synthesis half of the Web Speech API at all, at any version. A UC deployment targeting Android users needs to account for this if Firefox is a meaningful share of that population.

iOS requires a user gesture. On iOS Safari, `speak()` only works when called from inside a user-initiated gesture handler such as a tap. Called outside that context, the utterance is silently dropped, no error thrown, which makes this a debugging trap for anyone unaware of the constraint.

Voice availability is completely unpredictable. The exact voices, their quality, and which languages they cover depend entirely on the user's device and operating system. There is no standard voice set and no guarantee that any specific language has a local voice installed on a given machine. A UC deployment serving a specific language community cannot assume that language will sound good, or exist at all, on every visitor's device.

Still a draft specification. As of the most recent check, the governing document is a W3C Community Group Draft Report, not a finished standard. Behaviour can still shift between browser versions, and there is no formal conformance guarantee across implementations.

`getVoices()` timing is a known implementation trap. Voices often load asynchronously; calling `getVoices()` immediately on page load can return an empty list. Correct usage requires handling the `onvoiceschanged` event, a detail easy to miss and a common source of "no voices available" bug reports that are actually just a timing issue.

Not a substitute for screen-reader accessibility

This is the single most important caution in this entry, and it deserves to be stated plainly rather than buried in a table cell.

`speechSynthesis` reading page text aloud is not the same thing as a page being accessible to screen-reader users. Screen readers (JAWS, NVDA, VoiceOver, TalkBack) do far more than convert text to audio: they navigate by heading structure, announce ARIA roles and states, expose form labels, and let a user jump by landmark rather than listen linearly through an entire page. None of that comes from calling `speechSynthesis.speak()` on a block of text.

This distinction has real-world stakes: "read aloud" widgets built on this API are sometimes marketed by accessibility-overlay vendors as a substitute for genuine WCAG remediation, a substitution the disability community broadly rejects, and the comparison has been described directly as the same category of mistake as treating an automated scan as proof of accessibility.

For UC purposes: using `speechSynthesis` as the voice engine inside a purpose-built AAC tool like Cboard is a legitimate, well-fitted use. Using it to bolt a "listen to this page" button onto an otherwise inaccessible site, in place of fixing that site's semantic structure, is not accessibility work and should not be scored as such in any UC evaluation.

UC positioning

Adopt this API specifically for direct voice-output features, the way Cboard uses it: a purpose-built interaction where the spoken output is the product, not a bolted-on accessibility gesture. It is free, sovereign, offline-capable, and requires no infrastructure a UC deployment would need to maintain or pay for.

Do not adopt it as a screen-reader substitute, and do not let a UC evaluation of a tool that uses `speechSynthesis` count that usage toward the tool's actual accessibility score without separately verifying semantic HTML, ARIA, and keyboard navigation.

Do not extend this entry's Strong sovereignty rating to `SpeechRecognition`. That is a different interface, with a different, cloud-dependent story, and warrants its own separate radar entry if and when a UC deployment needs speech input rather than speech output.

See also

- MDN: Using the Web Speech API
- W3C Web Speech API Community Group Draft Report
- `aac-communication-cboard-v0-1-0.md` — the tool this entry's evaluation was prompted by

SAT radar entry v0.1.0. Reviewed 2026-08-07 against Universal Cake criteria: zero cost, disability accessibility, cross-platform device reach, multilingual support, and data sovereignty.